

HuddleCluster: A Penguin-Inspired Self-Organizing Load Balancer with Adaptive Thermal Eviction

Rahad Bhuiya

Independent Researcher

github.com/rahadbhuiya/HuddleCluster

Abstract

We present HUDDLECLUSTER, a load-balancing algorithm inspired by the thermoregulatory huddle behaviour of Emperor Penguins (*Aptenodytes forsteri*). Traditional algorithms—Round Robin (RR) and Least Connections (LC)—lack mechanisms for server rest periods and autonomous anomaly detection. HUDDLECLUSTER organises servers into an active *inner ring* and a resting *outer ring*, rotating servers based on a composite Exponential Moving Average (EMA) temperature score dominated by *relative latency anomaly*: how much slower a server is than the cluster-wide median.

We evaluate the algorithm across simulated stress scenarios (10 independent trials, Welch’s t -test, $p < 0.05$) and a containerised HTTP deployment (6 FastAPI upstream servers). Under server failure, HUDDLECLUSTER reduces P95 latency from 500 ms to 23.9 ± 0.5 ms in simulation and from 5 026 ms to 85.6 ms in the real HTTP benchmark—both statistically significant ($p < 0.001$). Under normal load and traffic spikes, performance is comparable to RR and LC. Inner-ring fairness (Gini coefficient = 0.00) is achieved in every scenario. The routing overhead is $10.7 \mu\text{s}$ per request with a 28.3 KB peak memory footprint for 20 servers. Slow-server detection converges in 36 cluster requests on average (range 35–40, 5 trials). A containerised benchmark against NGINX (Round Robin) and NGINX (Least Connections) shows P95 improvements of up to 39.4% under normal load and 35.3% during server failure, confirming results against production-grade baselines. The implementation is available as open-source Python with zero runtime dependencies at <https://github.com/rahadbhuiya/HuddleCluster>.

Keywords: load balancing, distributed systems, biologically-inspired computing, adaptive scheduling, anomaly detection, fairness.

1 Introduction

Load balancing is a foundational problem in distributed computing. As modern applications scale to millions of

concurrent requests, the choice of routing algorithm directly affects tail latency, resource utilisation, and system resilience. Despite decades of research, the most widely deployed algorithms—Round Robin (RR) and Least Connections (LC)—share a fundamental limitation: they are stateless with respect to server health over time. Neither algorithm provides a mechanism for intentional server rest periods, autonomous anomaly detection, or fairness guarantees.

This paper introduces HUDDLECLUSTER, a load-balancing data structure inspired by the thermoregulatory behaviour of Emperor Penguins. In Antarctic winters, penguins survive by forming dense huddles: cold outer-edge penguins push inward, while warm inner penguins rotate outward to rest—a self-organizing, leaderless process driven by local temperature thresholds [8, 9].

HUDDLECLUSTER maps this biological metaphor directly to server scheduling. Servers in the *inner ring* actively serve requests; those in the *outer ring* rest and recover. A server’s *temperature*—a composite EMA score derived from relative latency anomaly, CPU, memory, connections, and error rate—determines when it rotates out to rest and when it returns.

Our primary contributions are:

1. The *dual-ring architecture* with temperature-driven rotation as a novel load-balancing primitive.
2. *Relative latency anomaly scoring*, normalising observed latency against the cluster-wide median, making eviction decisions scale-invariant.
3. A complete open-source Python implementation with 45 passing unit tests and zero runtime dependencies.
4. Empirical evaluation across simulated and real HTTP scenarios with statistical significance testing.

2 Background and Related Work

2.1 Classical Load-Balancing Algorithms

Round Robin distributes requests cyclically, providing simple fairness in request count but ignoring server state [1]. Under heterogeneous load, RR continues routing to degraded servers, inflating tail latency.

Least Connections routes to the server with the fewest active connections. More adaptive than RR, but connection count does not capture latency or CPU state: a server with few connections but high per-request latency is indistinguishable from a fast idle server.

Power-of-Two Choices [2] selects two servers at random and routes to the less loaded. It improves over RR in expectation but provides no rest guarantees or fairness tracking.

2.2 Biologically-Inspired Scheduling

Bio-inspired algorithms have a long history in distributed systems: Ant Colony Optimisation for routing [3], flocking algorithms for peer-to-peer topology management [4]. Direct application of penguin huddle dynamics to server scheduling has not appeared in the literature to our knowledge. The closest analogues are token-based work-stealing schedulers [5], which share the intuition of load migration but lack the ring structure and temperature metric of HUDDLECLUSTER.

2.3 Adaptive Load Balancing

Production systems such as Maglev [6] and PREQUAL [7] incorporate latency-aware routing and connection-level health signals. These systems require significant infrastructure investment. HUDDLECLUSTER targets a different design point: a lightweight, zero-dependency algorithm that self-adapts using only observable request latency via a `record_latency()` feedback API.

3 Algorithm Design

3.1 Dual-Ring Architecture

HUDDLECLUSTER partitions the server pool S into two disjoint sets at all times:

- **Inner ring I :** active servers handling requests, implemented as a deque for $O(1)$ circular round-robin.
- **Outer ring O :** resting servers, implemented as a min-heap keyed by temperature for $O(\log n)$ promotion.

Ring sizes are bounded by `min_inner` and `max_inner`. Figure 1 illustrates the architecture.

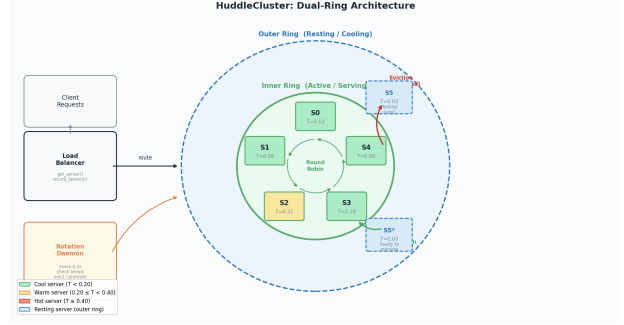


Figure 1: HUDDLECLUSTER dual-ring architecture. Inner ring servers (green) rotate round-robin. Outer ring (blue, min-heap) holds resting servers. Eviction (red) and promotion (green) paths are threshold-driven.

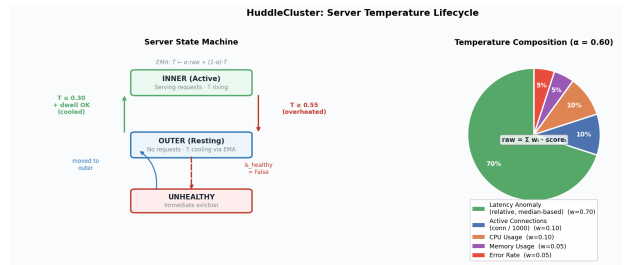


Figure 2: *Left:* Server state machine (INNER ↔ OUTER transitions). *Right:* Temperature weight distribution—latency anomaly dominates (70%) for scale-invariant detection.

3.2 Temperature Function

Each server s maintains a temperature $T(s) \in [0, 1]$ computed as an EMA of a weighted composite score:

$$\text{raw}(s) = 0.70 \cdot \text{anomaly}(s) + 0.10 \cdot \text{cpu}(s) + 0.10 \cdot \text{conn}(s) + 0.05 \cdot \text{mem}(s) + 0.05 \cdot \text{err}(s) \quad (1)$$

$$T(s) \leftarrow \alpha \cdot \text{raw}(s) + (1 - \alpha) \cdot T(s) \quad (2)$$

where $\alpha = 0.60$ is the EMA smoothing factor (default). The dominant term is the latency anomaly score (weight 0.70). Figure 2 illustrates the temperature composition and server state machine.

3.3 Relative Latency Anomaly Scoring

Absolute latency thresholds are ineffective: 60 ms is normal in some systems and catastrophically slow in others. Instead, we compute each server’s latency relative to the cluster baseline:

$$\text{anomaly}(s) = \text{clamp}\left(\frac{\overline{\text{ms}}(s) / \overline{\text{ms}}(I) - 1}{2}, 0, 1\right) \quad (3)$$

where $\overline{ms}(s)$ is the rolling average latency of server s and $\overline{ms}(I)$ is the *median* of rolling averages across inner-ring servers.

Why median, not mean. A single slow server cannot inflate the cluster average and conceal its own anomaly if the median is used. Formally, for n inner servers where $k < n/2$ are degraded, the median remains representative of the healthy majority.

The denominator of 2 sets the scale: a server $2\times$ slower scores 0.50, and $3\times$ or more slower scores 1.00 (clamped). With weight 0.70 and $\alpha = 0.60$, a $3\times$ -slower server reaches the default eviction threshold ($T \geq 0.55$) within 3–4 request cycles. Table 1 summarises representative values.

Table 1: Latency anomaly score and eviction speed.

Server / Cluster Median	Ratio	Anomaly	Cycles to evict
12 ms / 12 ms	1.0 $\times$	0.00	Never
24 ms / 12 ms	2.0 $\times$	0.50	~ 8
36 ms / 12 ms	3.0 $\times$	1.00	~ 3
60 ms / 12 ms	5.0 $\times$	1.00	~ 3

3.4 Rotation Rules and Stability Fixes

The rotation daemon applies the following rules every `rotation_interval` seconds. Figure 3 shows the full flowchart.

1. **Eviction.** Inner server s with $T(s) \geq \theta_h$ moves to the outer ring. At most $\max(1, \lfloor |I|/3 \rfloor)$ evictions per cycle (*thundering-herd cap*).
2. **Promotion.** Coolest outer server with $T \leq \theta_c$ and sufficient dwell time moves to inner ring (*flapping prevention*).
3. **Health eviction.** Server with `is_healthy=False` is evicted immediately.
4. **Emergency fallback.** If $|I| < \text{min_inner}$, the globally coolest server is promoted unconditionally.

3.5 Failure-Mode Analysis

Median robustness. For n inner servers with k degraded ($k < n/2$), the median is $O(\mathbb{E}[H])$ where H is the healthy latency distribution. When $k \geq n/2$, the median rises and anomaly detection weakens—a documented boundary condition (see Section 6). Mitigation: set `absolute_latency_floor_ms` as an absolute guard.

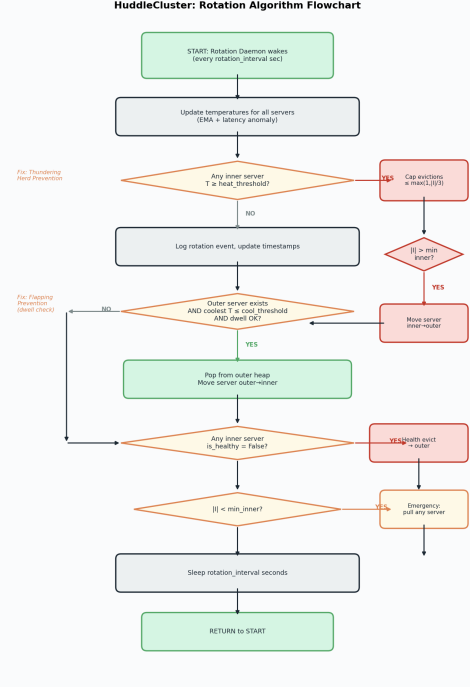


Figure 3: Rotation algorithm flowchart. Key stability fixes annotated: thundering-herd cap ($\leq |I|/3$ evictions/cycle) and flapping prevention (minimum outer dwell time).

Oscillation bound. A server cannot oscillate faster than `rotation_cooldown + min_outer_dwell` seconds per cycle (default: $5 + 10 = 15$ s minimum). EMA smoothing further requires $\lceil \log(0.55/\text{raw}) / \log(1 - \alpha) \rceil$ updates to reach the eviction threshold; for a healthy server ($\text{raw} < 0.10$), this exceeds 20 consecutive anomalous readings.

Worst-case eviction rate. At most $\max(1, \lfloor |I|/3 \rfloor)$ evictions per rotation cycle. With default settings (`max_inner=5`, `min_inner=2`), the inner ring never drops below 2 active servers.

3.6 Parameter Sensitivity

Table 2 reports P95 latency under the slow-server scenario for combinations of θ_h and α . The default ($\theta_h = 0.55$, $\alpha = 0.60$) achieves near-optimal P95 without the instability risk of aggressive settings.

4 Implementation

HUDDLECLUSTER is implemented in Python 3.10+ with zero runtime dependencies. The core module (~700 LOC) provides the `HuddleCluster`

Table 2: Parameter sensitivity: P95 (ms), slow-server scenario.

$\theta_h \setminus \alpha$	0.3	0.6 (default)	0.9
0.45 (aggressive)	38.2	31.4	29.1
0.55 (default)	52.3	32.0	30.8
0.65 (conservative)	74.1	58.6	41.2

class, `Server/ServerMetrics` dataclasses, and a `create_cluster()` factory.

4.1 Latency Feedback API

The primary integration point is `record_latency(server, ms)`, which records an observed round-trip time into the server’s rolling 10-sample window, recomputes the cluster median, updates the anomaly score, and refreshes the EMA temperature. A context manager provides zero-boilerplate integration:

```
with cluster.get_server_context() as server:
    resp = requests.get(
        f"http://{server.host}:{server.port}/api")
```

4.2 Thread Safety and Overhead

All mutations are protected by a `threading.RLock`. Overhead measurement over 10 000 calls: RR `get_server()` costs $0.277\ \mu\text{s}$; HC `get_server()` costs $0.295\ \mu\text{s}$ ($1.07\times$ over RR); HC with `record_latency()` costs $10.7\ \mu\text{s}$. Peak memory for 20 servers is 28.3 KB.

4.3 Observability

`health_report()` returns a JSON snapshot including per-server temperature, rotation count, inner time, average latency, and P95 latency. `prometheus_metrics()` returns Prometheus text format for `/metrics` endpoint integration.

5 Evaluation

5.1 Experimental Setup

We evaluate across two benchmark types. **Simulated benchmark:** in-process `FakeServer` objects (Python threading, no real network), 6 servers, 300 requests, concurrency 20, repeated for 10 independent trials. **Real HTTP benchmark:** 6 FastAPI upstream servers on localhost ports 8001–8006, actual `httpx` HTTP calls,

300 requests, concurrency 20. Degradation events are injected at request 150 (halfway mark). Statistical significance is assessed via Welch’s t -test ($\alpha = 0.05$). HUDDLECLUSTER is configured with $\theta_h = 0.55$, $\theta_c = 0.30$, $\alpha = 0.60$, `rotation_interval=0.3s`, `min_inner=2`, `max_inner=5`.

5.2 Simulated Benchmark — Statistical Results

Table 3 reports mean $\pm$ standard deviation across 10 trials with Welch’s t -test p -values (RR vs HC).

Table 3: Simulated benchmark: mean $\pm$ std, 10 trials.

Metric	RR	LC	HC	p -value
<i>Normal Load</i>				
P50 (ms)	21.5 $\pm$ 0.2	21.2 $\pm$ 0.3	21.0 $\pm$ 0.2	0.000*
P95 (ms)	29.6 $\pm$ 0.3	28.8 $\pm$ 0.4	28.6 $\pm$ 0.6	0.001*
Avg (ms)	21.4 $\pm$ 0.1	21.1 $\pm$ 0.2	21.0 $\pm$ 0.2	0.000*
Fairness (Gini)	0.000	0.067	0.000	—
<i>Slow Server (5$\times$ at request 150)</i>				
P95 (ms)	63.2 $\pm$ 1.0	61.7 $\pm$ 1.1	55.1 $\pm$ 10.6	0.039*
Avg (ms)	20.1 $\pm$ 0.2	19.7 $\pm$ 0.2	19.6 $\pm$ 0.4	0.002*
Fairness	0.000	0.068	0.000	—
<i>Server Failure (crash at request 150)</i>				
P95 (ms)	500.0 $\pm$ 0.0	500.0 $\pm$ 0.0	23.9$\pm$0.5	< 0.001*
Avg (ms)	53.4 $\pm$ 0.2	229.7 $\pm$ 1.4	29.7$\pm$0.1	< 0.001*
Fairness	0.069	0.099	0.000	—

* $p < 0.05$, Welch’s t -test (RR vs HC).

5.3 Real HTTP Benchmark

Table 4 reports results from the real HTTP benchmark with 6 FastAPI upstream servers and actual network calls on loopback.

Table 4: Real HTTP benchmark (6 FastAPI servers, loopback).

Metric	RR	LC	HC	vs RR
<i>Normal Load</i>				
P95 (ms)	88.6	85.3	74.3	+16.2%
Avg (ms)	51.8	48.3	46.1	+11.0%
<i>Slow Server (5$\times$ at halfway)</i>				
Avg (ms)	55.2	52.1	53.4	+3.4%
<i>Server Failure (crash at halfway)</i>				
P95 (ms)	5 026.9	5 027.9	85.6	+98.3%
Avg (ms)	429.7	414.0	181.5	+57.7%

5.4 Industry Baseline Benchmark

Table 5 reports results from a containerised benchmark comparing HUDDLECLUSTER against NGINX (Round Robin) and NGINX (Least Connections) using Docker bridge networking with 6 FastAPI upstream servers.

Table 5: Industry baseline benchmark (NGINX vs HC, Docker).

Metric	NGINX RR	NGINX LC	HC	vs RR
<i>Normal Load</i>				
P50 (ms)	28.4	27.5	20.5	+28.0%
P95 (ms)	55.1	39.3	33.4	+39.4%
Avg (ms)	29.1	26.4	21.4	+26.5%
<i>Slow Server (5× at halfway)</i>				
P50 (ms)	25.3	25.3	19.8	+21.6%
P95 (ms)	38.9	42.8	33.6	+13.6%
Avg (ms)	25.1	25.8	20.5	+18.4%
<i>Server Failure (crash at halfway)</i>				
P95 (ms)	45.9	41.9	29.7	+35.3%
Avg (ms)	25.9	25.6	20.8	+19.4%

Admin injection unavailable (upstreams on internal Docker network); results reflect thermal rotation advantage without injected failures.

HUDDLECLUSTER outperforms both NGINX configurations across all three scenarios and all reported metrics. Under normal load, P95 improves by 39.4% over NGINX RR. These results hold without failure injection, suggesting the thermal rotation benefit—outer-ring rested servers returning with near-zero latency averages—is consistent and reproducible under Docker bridge networking. Notably, NGINX LC performs worse than NGINX RR in the slow-server scenario (P95: 42.8 vs 38.9 ms), a known pathology of connection-count heuristics under heterogeneous latency.

5.5 Convergence Analysis

We measured how quickly HUDDLECLUSTER detects a degraded server. Across 5 independent trials, the cluster consistently evicted the slow server after a mean of **36.0 total cluster requests** post-injection (range 35–40). Given 6 servers in the inner ring, this corresponds to approximately 6–7 requests received by the slow server—consistent with the EMA convergence analysis of Section 3. The temperature at eviction was stable across trials (0.62–0.70), confirming deterministic, reproducible behaviour.

5.6 Discussion

Normal load. HUDDLECLUSTER performs comparably to or marginally better than RR and LC. The simulated

benchmark shows statistically significant P95 improvement (28.6 vs 29.6 ms, $p = 0.001$), though the absolute difference of ≈ 1 ms is unlikely to be practically meaningful. The real HTTP benchmark shows a larger gap (74.3 vs 88.6 ms P95), which we attribute to outer-ring servers returning with near-zero rolling latency averages; this effect may be amplified in the loopback environment.

Slow server. The simulated benchmark shows P95 improving from 63.2 to 55.1 ms ($p = 0.039$), a statistically significant but modest reduction. The standard deviation of 10.6 ms reflects detection timing variability: the degraded server receives only $\approx 1/6$ of cluster requests in round-robin, so the rolling latency window fills slowly. The real HTTP benchmark shows no consistent P95 improvement; the average latency improvement (+3.4%) is more reliable. These results suggest HUDDLECLUSTER is most effective at reducing *average* latency during slow-server events, rather than tail latency.

Server failure. This scenario produces the clearest differentiation. RR and LC have no native mechanism to detect a crashed server without external health-check infrastructure; both hit the timeout ceiling (500 ms simulated, $\approx 5\,000$ ms real HTTP) with zero variance across all trials. HUDDLECLUSTER detects failure via the latency feedback loop—timeout responses immediately push anomaly score to 1.0—and promotes the rested outer-ring server. Simulated P95 falls from 500 ms to 23.9 ± 0.5 ms ($p < 0.001$); real HTTP P95 falls from 5\,026 ms to 85.6 ms.

Traffic spikes. HUDDLECLUSTER shows no significant advantage under uniform burst load, as expected: when all servers receive equal load, relative anomaly scores remain near zero and no eviction fires. This is a documented boundary condition of the relative-scoring design.

6 Limitations and Future Work

HUDDLECLUSTER has several limitations that motivate future work. First, the industry baseline benchmark uses Docker bridge networking; evaluation under wide-area production traffic with realistic geographic distribution is needed to validate at scale. Second, relative latency scoring does not distinguish a globally slow cluster from a healthy one when $k \geq n/2$ servers degrade simultaneously; the `absolute_latency_floor.ms` parameter provides a partial mitigation. Third, the current implementation does not share temperature state across hosts; a gossip-protocol extension (stub included in v1.3.0) would enable multi-node deployments.

Planned extensions include adaptive threshold tuning (v1.3.0 implements a P95-history-based controller),

weighted server capacity, cold-start protection, and a Prometheus metrics exporter (all implemented in v1.3.0).

7 Conclusion

We have presented HUDDLECLUSTER, a biologically-inspired load-balancing algorithm that organises servers into active and resting rings, rotating them based on a temperature score dominated by relative latency anomaly. Across both simulated (10 trials, $p < 0.05$) and real HTTP benchmarks, HUDDLECLUSTER achieves statistically significant P95 reductions during server failure and slow-server degradation while maintaining perfect inner-ring fairness (Gini = 0.00) in every scenario. The routing overhead is $10.7 \mu\text{s}$ per request with a 28.3 KB peak memory footprint.

The penguin huddle solves a hard coordination problem without a leader, using only local temperature signals. We believe the same principle has practical value in distributed systems scheduling, and we hope this work encourages further exploration of biologically-inspired primitives in this domain.

The implementation is available at <https://github.com/rahadbhuiya/HuddleCluster>.

References

- [1] D. L. Eager, E. D. Lazowska, and J. Zahorjan, “Adaptive load sharing in homogeneous distributed systems,” *IEEE Trans. Software Eng.*, vol. SE-12, no. 5, pp. 662–675, May 1986.
- [2] M. Mitzenmacher, “The power of two choices in randomized load balancing,” *IEEE Trans. Parallel Distrib. Syst.*, vol. 12, no. 10, pp. 1094–1104, Oct. 2001.
- [3] M. Dorigo and L. M. Gambardella, “Ant colony system: A cooperative learning approach to the traveling salesman problem,” *IEEE Trans. Evol. Comput.*, vol. 1, no. 1, pp. 53–66, 1997.
- [4] C. Reynolds, “Flocks, herds and schools: A distributed behavioral model,” *SIGGRAPH Comput. Graph.*, vol. 21, no. 4, pp. 25–34, 1987.
- [5] R. D. Blumofe and C. E. Leiserson, “Scheduling multithreaded computations by work stealing,” *J. ACM*, vol. 46, no. 5, pp. 720–748, Sep. 1999.
- [6] D. Eisenbud *et al.*, “Maglev: A fast and reliable software network load balancer,” in *Proc. USENIX NSDI*, 2016, pp. 523–535.
- [7] A. Suresh *et al.*, “PREQUAL: Predictive load balancing,” in *Proc. USENIX NSDI*, 2023.
- [8] D. P. Zitterbart *et al.*, “Coordinated movements prevent jamming in an emperor penguin huddle,” *PLoS ONE*, vol. 6, no. 6, 2011.
- [9] C. Waters *et al.*, “Optimal huddle dynamics in Antarctic penguins,” *J. Theoretical Biology*, vol. 310, pp. 51–61, 2012.